

Sistema Integrado de Administración (CEP)

MANUAL TÉCNICO INTEGRAL

Documentación Completa: Arquitectura, Despliegue y Código



**CONTRALORÍA
DEL ESTADO PORTUGUESA**

Índice de Contenidos

Capítulo 1: Arquitectura y Requerimientos Generales

Capítulo 2: Base de Datos, Trazabilidad y Aislamiento

Capítulo 3: Transacciones Financieras y Módulos Complejos

Capítulo 4: Motor de Nómina y Procesamiento en Segundo Plano

Capítulo 5: Despliegue y Mantenimiento

Capítulo 6: Requerimientos del Sistema

Capítulo 1: Arquitectura y Requerimientos Generales

1.1 Stack Tecnológico

El sistema está desarrollado sobre Laravel (PHP) con las siguientes tecnologías:

- Backend: PHP 8.x, Framework Laravel 10/11
- Base de Datos: MySQL / MariaDB
- Frontend: Blade Templates, TailwindCSS, Alpine.js
- Dependencias críticas: Spatie Permission (Roles), DomPDF (Generación de Documentos).

1.2 Patrón MVC y Repository / Service

Se utiliza el patrón MVC (Modelo-Vista-Controlador). Además, el sistema separa las consultas puras a la BD de la lógica de negocio usando Servicios. Por ejemplo, en el catálogo de partidas presupuestarias se emplea la caché.

Clase CatalogoCache (app/Services/CatalogoCache.php)

Este servicio almacena en caché las partidas para evitar colapsar la base de datos cada vez que se carga un formulario de creación:

```
public static function partidas() {  
    return Cache::remember("catalogo_partidas_" . session("ejercicio_id"), 3600,  
function () {  
    return PartidaPresupuestaria::where("ejercicio_fiscal_id",  
session("ejercicio_id"))->get();  
    });  
}
```

1.3 Optimización de Consultas (Eager Loading)

Para resolver problemas críticos de rendimiento (N+1 queries), los controladores obligan a cargar relaciones anidadas usando el método with(). En el modelo Compromiso.php, esto está configurado nativamente:

```
protected $with = ["partida"];
```

Esto asegura que cada vez que se consulte un compromiso, su partida y saldo se anexen en memoria de forma inmediata.

Capítulo 2: Base de Datos, Trazabilidad y Aislamiento

2.1 Trazabilidad (Auditoría)

La mayoría de los modelos críticos implementan el paquete "owen-it/laravel-auditing". Esto permite registrar quién, cuándo y qué campos modificó en un registro (ej. al anular un compromiso).

2.2 Aislamiento de Datos: Trait FiltraPorEjercicio

Para que un usuario trabajando en el Ejercicio Fiscal 2026 no modifique por error datos del 2025, se implementó un Global Scope en `app/Traits/FiltraPorEjercicio.php`.

Funcionamiento interno:

Laravel intercepta la consulta SQL a nivel del ORM y añade automáticamente la cláusula `WHERE ejercicio_fiscal_id = [ID_SESION]` en todos los modelos que importen este Trait (Ej. Compromiso, Causacion, Nomina).

```
protected static function bootFiltraPorEjercicio() {
    static::addGlobalScope("ejercicio", function (Builder $builder) {
        if (session()->has("ejercicio_id")) {
            $builder->where($builder->getQuery()->from . ".ejercicio_fiscal_id",
            session("ejercicio_id"));
        }
    });
}
```

Capítulo 3: Transacciones Financieras y Módulos Complejos

3.1 Atomización de los Pagos (Evitando descuadres)

En CEP, pagar un Compromiso o una Nómina es un evento altamente crítico. Si se emite el pago pero la base de datos se cuelga antes de descontar el saldo de la Partida Presupuestaria, la institución pierde la cuadratura financiera.

Solución: Transacciones DB (NominaController.php @ pagar):

```
DB::transaction(function () use ($request, $nomina) {  
    // 1. Obtener la partida  
    $partida = PartidaPresupuestaria::findOrFail($request->partida_id);  
    // 2. Validación estricta  
    if ($nomina->total_neto > $partida->saldo_actual) { throw new \Exception("Saldo insuficiente"); }  
    // 3. Descontar saldo y registrar movimiento en bitácora  
    $partida->decrement("saldo_actual", $nomina->total_neto);  
    MovimientoPartida::create([...]);  
    // 4. Marcar pagada  
    $nomina->update(["estado" => "pagada"]);  
});
```

Si cualquiera de estos pasos falla, la transacción se deshace (Rollback), protegiendo la integridad del erario.

3.2 Generación de Secuenciales (Prevención de Duplicidad)

Al emitir comprobantes (COM-2026-0001), se requiere evitar que dos usuarios generen el mismo número simultáneamente. Se usa max("id") sobre la base de datos en lugar de hacer loops ineficientes en PHP (Compromiso::generarNumero).

3.3 Cálculo de Retenciones

El modelo "Retencion.php" contiene una función dinámica "calcularMonto(\$totalConIva)". Extrae la base imponible si se trata del IVA, o aplica la tasa porcentual / monto fijo según esté configurado en la base de datos.

3.4 Almacén (Manejo de Stock)

Al aprobar una Recepción, un Observer/Servicio suma las cantidades al inventario central. Los despachos restan de ese stock, asegurando que no se registren valores negativos.

Capítulo 4: Motor de Nómina y Procesamiento Asíncrono

4.1 Jobs y Queues

El cálculo de la nómina recorre cientos de empleados, aplicando fórmulas de conceptos y deduciendo retenciones fiscales complejas. Para evitar que el navegador muestre "Timeout", la ejecución matemática se encola.

// NominaController.php @ store

```
CalcularNominaJob::dispatch($nomina, $idsRetenciones);
```

El usuario es redirigido inmediatamente al panel con un estado de "Borrador/En proceso", mientras en el servidor un "queue:worker" evalúa el Job, iterando el modelo Empleado y generando miles de registros NominaDetalle silenciosamente.

Capítulo 5: Despliegue y Mantenimiento

5.1 Instalación (Deploy)

Comandos típicos para preparar el entorno de producción en el servidor:

1. `composer install --no-dev --optimize-autoloader`
2. `npm run build`
3. `php artisan migrate`
4. `php artisan db:seed --class=RolesAndPermissionsSeeder`
5. `php artisan db:seed --class=DatosBaseSeeder`
6. `php artisan db:seed --class=PruebaIntegralSeeder`
7. `php artisan optimize:clear`

Capítulo 6: Requerimientos del sistema

Componente	Versión Mínima	Notas
• PHP	8.3	Con extensiones: ext-zip, ext-gd, ext-dom, ext-xml, ext-json
• Composer	2.x	Gestor de dependencias PHP
• MySQL / MariaDB	8.0 / 10.6	Base de datos principal
• Node.js	18+	Para compilar los assets CSS/JS
• npm	9+	Incluido con Node.js
• Servidor Web	Apache o Nginx	(Laragon incluye ambos)

(PHP, MySQL, MariaDB, npm, Apache y Nginx Vienen incluidos en el gestor de Base de Datos Laragon)